

Agentic AI: Architecture, Memory, and Workflows

An In-Depth Guide to Modern Autonomous AI Systems

Chapter 1: Introduction to Agentic AI

Agentic AI refers to autonomous systems driven by Large Language Models (LLMs) capable of perceiving environments, formulating goals, executing multi-step reasoning, calling external tools, and reflecting on outcomes. Unlike static text generators, agentic systems act independently to complete complex objectives.

Chapter 2: The ReAct Framework (Reasoning and Acting)

ReAct (Reasoning + Acting) is a fundamental architectural paradigm where an AI agent alternates between explicit thought steps (Reasoning) and action steps (Executing API calls or tools). In ReAct, the agent analyzes current observation, produces a internal monologue reasoning trace, selects an appropriate tool, receives the observation output, and iterates until the user request is completely resolved.

Chapter 3: Agent Memory Systems

Agent memory is divided into three primary categories: Short-term memory maintains active context within the LLMs immediate window. Working memory maintains state across active execution loops. Long-term memory utilizes vector databases to store, retrieve, and recall episodic past interactions, domain documents, and user preferences over extended periods.

Chapter 4: Tool Calling & External Capabilities

Tool calling enables LLMs to interface with real-world dynamic tools such as web search engines, Python code interpreters, SQL databases, REST APIs, and file systems. Tools extend an LLMs static internal knowledge into real-time operational execution.

Chapter 5: Planning Agents and Task Decomposition

Planning agents decompose complex high-level goals into ordered execution graphs or step-by-step sub-tasks. Planning strategies include Tree of Thoughts (ToT), Self-Reflection (Reflexion), and Graph-based task dependencies.

Chapter 6: LangGraph & Stateful Multi-Agent Systems

LangGraph is a stateful orchestration framework for building complex agentic systems as graphs. Nodes represent functions or LLM calls, while edges govern execution flow, conditional routing, and state transitions. LangGraph supports cyclic execution, checkpointing, multi-agent collaboration, and strict state management.